

StructZone: Sanitizing Structs Using Redzones

Alex Feenstra
VU Amsterdam

Joachim Bose
VU Amsterdam

Robert Klink
VU Amsterdam

Abstract—With the prevalence of memory unsafe languages such as C and C++, sanitizers are a popular choice for detecting vulnerabilities. However, most popular sanitizers do not support intra-struct overflow detection as this typically breaks the Application Binary Interface (ABI). We present StructZone, a sanitizer that detects overflow in structs using redzones, whilst attempting to preserve ABI compatibility through deflation of the structs or adaptation of library functions. Our evaluation shows that StructZone can successfully detect overflows, though with a significant overhead and with some compatibility issues on larger programs.

1. INTRODUCTION

Although many memory safe languages are increasingly popular [1], unsafe languages such as C and C++ still remain widely used in lower-level or legacy applications. However, the control over the memory provided by these languages is a double-edged sword, as the performance gains come with the increased risk of programmer mistakes introducing undefined behavior. In practice, this often goes wrong, leading to crashes, bugs, and serious security vulnerabilities.

Many methods exist that attempt to detect or mitigate such vulnerabilities. Some examples are static analysis, and sanitizers such as Asan [2], which detect vulnerabilities at runtime by placing poisoned memory areas around allocations called redzones.

However, existing sanitizers usually do not support structs, as the elements inside structs are allocated contiguously in memory, leaving no space to insert redzones unless the memory layout of the struct is modified with additional space (struct inflation). Modifying this struct layout leads to the Application Binary Interface (ABI) being broken, which makes it hard to use for existing libraries. Because libraries expect the struct laid out according to the ABI, we must remove the redzones (deflate) before handing the struct to such a library.

To investigate this gap, we introduce StructZone, a sanitizer designed specifically to detect memory safety violations within structs to find a middle ground between deflation and adaption for ABI compatibility. This is achieved with a combination of an instrumentation and inflation pass with a runtime library to manage the redzone checks. We evaluate the vulnerability detection of StructZone using example programs with memory overflows in structs, and evaluate the speed against a non-StructZone sanitized baseline to measure the overhead introduced by the sanitization. Using this, we aim to show the feasibility of struct sanitization whilst also discussing some of the limitations.

2. DESIGN

To detect errors in structs, we chose to add redzones in between struct members, which we initialize on allocation and remove on deallocation. We chose not to use any quarantines, as we are looking to detect spatial errors. We chose to use redzones over (another common method) fat-pointers as it is the industry standard for detecting overflows for several reasons:

- 1) Hardware accelerated memory tagging is not available yet for x86 chips due to AMD’s UAI and intel LAM development complications [3].
- 2) Identity-based memory sanitation is already able to track memory, but if it is not hardware accelerated, it is quite slow [4].
- 3) Identity-based memory sanitization for structs has already been done before, and we would like to do something new.

Struct zone adds redzones by instrumenting code with a compiler transformation pass which goes through (roughly) the following phases:

- 1) Create new (inflated) struct types.
- 2) Create function headers with inflated argument & return types (see Section 2-C).
- 3) Replace struct allocations and accesses to be the new struct types. (see Section 2-A)
- 4) Instrument the (de)allocations to add/remove redzones and accesses to check for redzones. (see Section 2-B)
- 5) Populate the deflation and inflation functions in phase 1.

A. Struct Updating

In phase 1, for each struct type, we allocate space between every member to poison later. During phase 3 we update all references to instead point to the updated structs, which must also be adjusted to account for the new memory layout. This requires rewriting pointer arithmetic, field offsets, and adjusting memory allocation sizes to reflect the increased footprint of the inflated structs, for which pointer analysis is used.

An inherent limitation is the practical complexity of updating all relevant instructions to accommodate the inflated structs. As programs grow bigger, they obtain more complicated interactions on the structs and between the instructions on the struct. Each of these cases requires explicit support, and some scenarios require substantial effort to handle correctly.

B. Redzone Access Detection

To detect redzones, we use a self-balancing binary tree (AVL tree), where redzones are inserted upon struct allocation and looked up when accessing memory. We can then quickly find the closest corresponding redzone with a complexity of $\mathcal{O}(\log(n))$, where n is the number of redzones. We optimize for lookup rather than for redzone insertion, as we expect allocation to rarely happen in hot code rather than memory accesses, which we expect to happen very often.

To mark a memory area as a redzone, instrumentation needs to be added whenever a struct is allocated. These allocations can happen in 3 places: (1) stack memory, (2) heap memory, and (3) global memory. In the (1) stack case, this is trivial, as we can simply insert redzone registering redzones at the start of a function. In the case of the heap, this is less easy, and we assume that the programmer uses the Libc-provided malloc family of functions. This means that custom allocators will not be detected, but adding support for these falls outside our scope. While we were not able to add support for global due to time constraints, doing so is not too difficult; we can make use of the global constructors to add redzones to them when the program starts¹.

To detect all overflows, we need to instrument *any* memory access with a redzone check. Due to time constraints, we choose to add the redzone check in a runtime library, which gets linked to the sanitized program. This simplifies the instrumentation to adding a single function call. This could be further optimized by setting the redzone to a specific byte checked before doing the AVL tree walk.

C. Conforming to the ABI

When calling a function from a library not compiled by our sanitizer, we need to fix up the struct redzones as the other library expects any arguments to not have struct redzones (deflated). Additionally, we need to also inflate return values (with struct redzones). The reverse operation needs to happen for any exported functions when compiling a library, as library users expect the functions they use to be deflated.

More precisely, we make a deflated and inflated function for every pre-existing function (externals and internals). For an external function like *stat()*, we make *stat.inflated()* which would inflate arguments, call the original, and deflate any returned data. Then for internal functions, we copy (and transform as described in Section 2-A) to an inflated shadow and make the original function inflate any arguments, call the inflated function, and deflate any returned values.

To make an inflator or a deflator for a struct, we allocate a copy of the deflated or inflated (respectively) temporarily, where we copy all fields into. If we are inflating or deflating a pointer to a struct for an argument to a function, we also mark this struct to be written back to its original memory place. This is important in functions like *stat()*, where the argument is a

¹We could also use the global destructors to remove them when the program ends, but since this does not alter anything, it is unnecessary.

pointer to a struct where the data gets written, which would get lost if we did not copy the data back.

Some complications start arising when data is passed by reference and the called function modifies this data. When the struct currently being handled does not have pointers to modified objects, the fields can be simply copied. If the called function changed the value of a pointer to a complex data structure we need to deflate too, it becomes really hard to make a temporary allocation for below our current stack frame.

3. IMPLEMENTATION

To implement the outlined design, we used LLVM 14 to write a transformation pass on LLVM IR. Additionally, we also created a runtime library which provides the tracking of the redzones, and the functions that check for redzone violations. To run StructZone on a program, `clang`, `opt`, and `llvm-link` are used in a pipeline to transform one or more C files into a StructZone sanitized program. For the sake of reproducibility, we provide a Makefile, Dockerfile, and docker compose orchestration file that builds the relevant tooling needed to compile with StructZone.

We used LLVM-14 as it has pointer types which were removed from newer versions of LLVM. With pointer types, we can find more information about what struct is in the memory which is pointed to by the pointer. This turns out to be especially crucial in our attempt to make a deflation layer.

During implementation, we encountered some additional challenges we wanted to highlight:

A. Finding and Updating heap allocations

To find heap allocations, we use bitcasts. We assume heap allocations in IR returns an opaque pointer which is always bitcast to the ‘correct’ type before use. So, if a bitcast’s source operand is a call to ‘malloc’, ‘calloc’ or ‘realloc’, we update the allocation size and put the redzones in the AVL-tree.

B. Phi nodes

Phi nodes are the most complex instruction to deal with. Like ICmp or Store instructions, their type is the type of their operands. However, a Phi node can have itself in its use-def chain. This is a problem as LLVM would assert when changing any of the phi instructions. To solve this, we introduce a bitcast to the inflated type. By then replacing all uses of the Phi node with the bitcast, we break the chain and can freely update the phi node.

4. EVALUATION

In order to evaluate our implementation, we discuss how well it is compatible with different kinds of programs and then go on to measure the overhead it introduces.

A. Compatibility

We have created a unit test suite with toy programs. These tests are either ‘safe’ tests, indicating that the sanitized program should succeed, or ‘overflow’ tests, indicating that the sanitized program should abort. In case of the ‘safe’ tests, we also compare the output to an unsanitized run to verify no semantics are altered.

Our unit tests currently verify:

- 1) Overflows within a struct (on the heap/stack),
- 2) Overflows between structs in an array (on the heap/stack),
- 3) Overflows in a nested struct type,
- 4) Overflows on structs created in another functions,
- 5) Support for self referential structs,
- 6) Support for re-allocated memory,
- 7) Support for function pointers,
- 8) Support for phi nodes,
- 9) Support for the stat syscall.

Larger programs are not always compatible. We have attempted to run a particular benchmark from SPEC, but we ran into some particular phi node chain (where multiple phi nodes formed a single chain) that our sanitizer pass could not handle. In general, it seems to be difficult to predict what instructions will be present in larger programs and in which ways they need to be handled.

In particular, many programs also seem to use unions; we have not figured out a way to deal with those in the general case. This is because if a union includes a struct type as one of its options, we would need to (de)register redzones based on the type that the union is accessed as, which comes down to points-to analysis.

B. Benchmark

We have written our own benchmark program to test the overhead of the sanitization. In essence, it calculates a formula consisting of addition, subtraction or multiplication. This is stored as a linked list, where each node is a struct containing a value, an operator and a pointer to the next node.

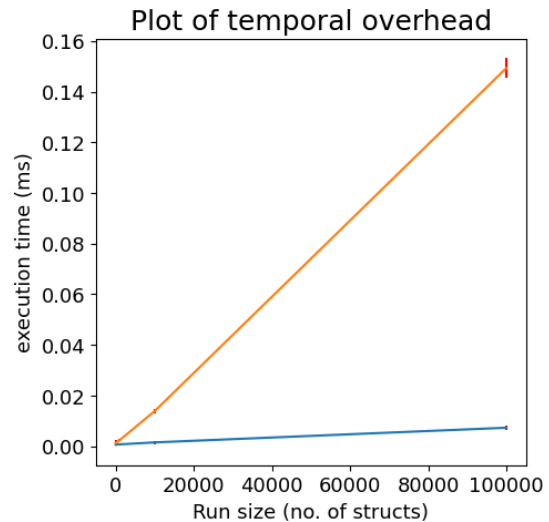
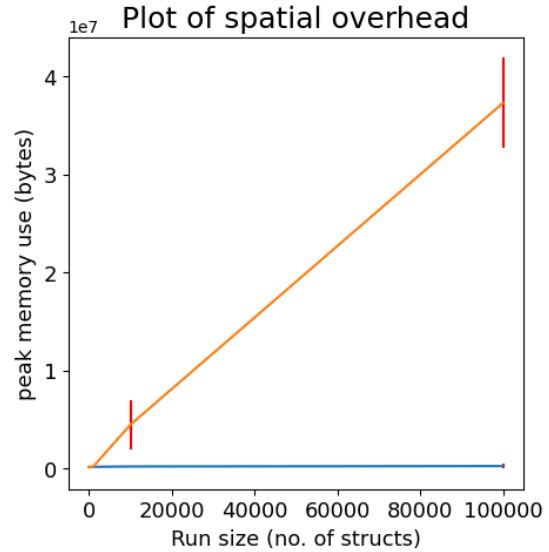
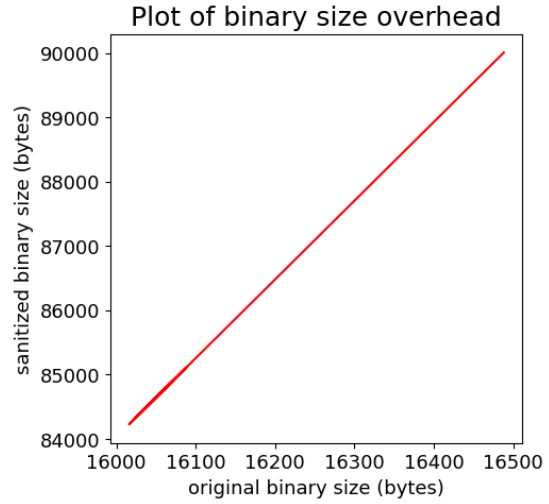
The program takes as CLI arguments the number of nodes the linked list should be, as well as a random seed value to keep the output reproducible.

We have run a number of input sizes for 1000 runs and report the results below:

Size	Original time (ms)	Sanitized time (ms)	Overhead
10	0.5688	1.1955	211%
10 ²	0.6315	1.3944	221%
10 ³	0.7208	2.3637	328%
10 ⁴	1.4979	13.8819	927%
10 ⁵	7.2841	149.3062	2049%

This seems to scale quite poorly, but this is not entirely unexpected. As the input size increases, so does the number of redzones *and* the number of memory accesses. This coupling accounts for the high increase in overhead we observe.

For visualization purposes, we have also provided several plots:



The data these plots are based on can be found in ‘benchmark/benchmark_out’.

We expect that the high standard deviation on the spatial overhead is due to the fact that we measure *peak* memory usage, which can mask fluctuations on lower run sizes.

Additionally, the binary size measurement shows a linear overhead. The linear trend is expected; we need additional code around each memory access, and that roughly scales with binary size. However, there is a notable increase of roughly 60 Kilobytes of binary size that we attribute to the runtime that is always linked.

C. Software specifications

- 1) Python 3.10.12
- 2) LLVM 14.0.0
- 3) GCC 11.4.0
- 4) GNU Make 4.3
- 5) Ubuntu 22.04.1 LTS (VM)
- 6) Virtualbox 6.1
- 7) Windows 11 Pro (Host OS)

D. Platform specifications

- 1) CPU: Ryzen 9 7950X3D; VM is assigned one core
- 2) RAM: 128GB DDR5 at 3.6GHZ; VM is assigned 16GB

5. DISCUSSION

Summary of technical limitations Struct zone struggles to run on real programs due to the following obstacles:

- StructZone cannot handle library calls where a pointer is returned (in any way) by the called function.
- Large programs with complex data flow graphs and global structs can break our implementation.
- StructZone cannot deal with unions.
- Functions must exit using the return instruction.
- Structs in global memory are not supported.
- Casting pointers to incorrect types can break our implementation.

Compiler passes and traversals When working with a compiler pass, the order of operations may not immediately seem to matter at first, but it is often highly important to avoid LLVM assertions and transformation mistakes.

Research versus Technical work All though this project went somewhere unexplored, it leaned too much to the coding side. We could have made this work fit better into the field if we looked around at other papers a little more before starting.

Quality of benchmarking There are some benchmarking crimes [5] that we committed. We will briefly discuss them:

- B1 We wrote our own (micro)-benchmark, because we could not get SPEC to work. It could well be the case that certain overhead is missed because our benchmark is very specific.
- D2 We compare against the baseline (unsanitized), but do not compare against other alternatives such as arm MTE. This means it is difficult to draw conclusions on how our solution performs with respect to the current state of the art.
- E3 Our unit tests suite show a number of cases where positives and negatives are correctly classified, as well as a few that are not due to time constraints, but there are cases (for instance, an arbitrary write exploit) that are not included.

6. CONCLUSION

We presented StructZone, a location based memory sanitizer for structs. We implemented the design and got many toy examples working. In our experiments, we found our implementation to get from hundreds to thousands percent overheads and poor scaling. We recommend to use identity based methods of sanitization when testing programs with many structs as we expect them to have less limitations.

7. CODE UPLOAD

The code is available under MIT license on github: <https://github.com/R0mbertus/StructZone.git>.

REFERENCES

- [1] D. urev, "Popularity of programming languages," *AIDASCO Reviews*, vol. 2, no. 2, p. 24–29, Dec. 2024.
- [2] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "Addresssanitizer: A fast address sanity checker," in *Proceedings of the 2012 USENIX conference on Annual Technical Conference*, vol. 12. Boston MA: Usenix, Jun. 2012, p. 28. [Online]. Available: <https://dl.acm.org/doi/10.5555/2342821.2342849>
- [3] M. Hertogh, S. Wiebing, and C. Giuffrida, "Leaky address masking: Exploiting unmasked spectre gadgets with noncanonical address translation," in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024, pp. 3773–3788.
- [4] E. Q. Vintila, P. Zieris, and J. Horsch, "Evaluating the Effectiveness of Memory Safety Sanitizers," in *2025 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2025, pp. 774–792. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/SP61157.2025.00088>
- [5] E. van der Kouwe, D. Andriesse, H. Bos, C. Giuffrida, and G. Heiser, "Benchmarking crimes: An emerging threat in systems security," *CoRR*, vol. abs/1801.02381, 2018. [Online]. Available: <http://arxiv.org/abs/1801.02381>